

6. СТРУКТУРА ПРОЕКТА (PROJECT STRUCTURE)

Эта глава предлагает некоторые идеи по структурированию ваших проектов на Rust. Для простых проектов структура, создаваемая командой `cargo new`, скорее всего, будет тем, о чём вы почти не задумываетесь. Возможно, вы добавите несколько модулей, чтобы разбить код, и несколько зависимостей для дополнительной функциональности, но на этом, пожалуй, всё. Однако по мере того как проект растёт в размере и сложности, вы обнаружите, что вам нужно выйти за эти рамки. Возможно, время компиляции вашего крейта выходит из-под контроля, или вам нужны условные зависимости, или нужна более удачная стратегия непрерывной интеграции. В этой главе мы рассмотрим некоторые инструменты, которые предоставляют язык Rust и в особенности Cargo, чтобы упростить управление подобными вещами.

Feature-флаги (Features)

Feature-флаги (features) — это основной инструмент Rust для кастомизации проектов. По своей сути feature-флаг — это просто флаг сборки, который крейты могут передавать своим зависимостям, чтобы добавить опциональную функциональность. Feature-флаги сами по себе не несут никакого семантического смысла — вместо этого *вы* решаете, что означает тот или иной feature-флаг для *вашего* крейта.

В общем случае мы используем feature-флаги тремя способами: чтобы включать опциональные зависимости, чтобы условно подключать дополнительные компоненты крейта и чтобы расширять поведение кода. Обратите внимание, что все эти применения являются *аддитивными*: feature-флаги могут добавлять функциональность к крейту, но, как правило, не должны делать таких вещей, как удаление модулей или замена типов или сигнатур функций. Это вытекает из принципа, что если разработчик вносит простое изменение в свой `Cargo.toml`, например добавляет новую зависимость или включает feature-флаг, это не должно приводить к тому, что его крейт перестаёт компилироваться. Если у крейта есть взаимоисключающие feature-флаги, этот принцип быстро рушится: если крейт А зависит от одного feature-

флага крейта C, а крейт B — от другого, взаимоисключающего feature-флага C, то добавление зависимости на крейт B сломает крейт A! По этой причине мы обычно следуем принципу: если крейт A компилируется с крейтом C при некотором наборе feature-флагов, он также должен компилироваться, если на крейте C включены *все* feature-флаги.

Cargo довольно сильно опирается на этот принцип. Например, если два крейта (A и B) оба зависят от крейта C, но каждый из них включает на C разные feature-флаги, Cargo соберёт C только один раз, со *всеми* feature-флагами, которые требуются хотя бы одному из A или B. То есть он возьмёт объединение запрошенных feature-флагов для C по A и B. Из-за этого, как правило, трудно добавить взаимоисключающие feature-флаги в крейты Rust: велики шансы, что какие-то два зависимых крейта будут зависеть от вашего крейта с разными feature-флагами, и если эти feature-флаги взаимоисключающие, нижестоящий крейт не соберётся.

ПРИМЕЧАНИЕ Я настоятельно рекомендую настроить инфраструктуру непрерывной интеграции так, чтобы проверять, что ваш крейт компилируется при любой комбинации его feature-флагов. Один из инструментов, который помогает в этом, — cargo-hack, который можно найти по адресу <https://github.com/taiki-e/cargo-hack>.

Определение и подключение feature-флагов (Defining and Including Features)

Feature-флаги определяются в Cargo.toml. Листинг 6-1 показывает пример крейта с именем foo с простым feature-флагом, который включает опциональную зависимость syn.

```
[package]
name = "foo"
...
[features]
derive = ["syn"]

[dependencies]
syn = { version = "1", optional = true }
```

Листинг 6-1: Feature-флаг, который включает опциональную зависимость

Когда Cargo компилирует этот крейт, он по умолчанию не будет

компилировать крейт `syn`, что сокращает время компиляции (зачастую существенно). Крейт `syn` будет скомпилирован только в том случае, если какому-то нижестоящему крейту нужны API, включаемые feature-флагом `derive`, и он явно подключит этот флаг. Листинг 6-2 показывает, как такой нижестоящий крейт `bar` включил бы feature-флаг `derive` и тем самым подключил зависимость `syn`.

```
[package]
name = "bar"
...
[dependencies]
foo = { version = "1", features = ["derive"] }
```

Листинг 6-2: Включение feature-флага зависимости

Некоторые feature-флаги используются настолько часто, что имеет больше смысла, чтобы крейт от них отказывался, а не подключал их. Чтобы это поддержать, Cargo позволяет определять набор feature-флагов по умолчанию (default features) для крейта. И аналогично он позволяет отказаться от feature-флагов по умолчанию у зависимости. Листинг 6-3 показывает, как `foo` может сделать свой feature-флаг `derive` включённым по умолчанию, при этом также отказываясь от некоторых feature-флагов `syn`, заданных по умолчанию, и вместо этого включая только те, которые нужны ему для feature-флага `derive`.

```
[package]
name = "foo"
...
[features]
derive = ["syn"]
default = ["derive"]

[dependencies.syn]
version = "1"
default-features = false
features = ["derive", "parsing", "printing"]
optional = true
```

Листинг 6-3: Добавление feature-флагов по умолчанию и отказ от них, а также опциональных зависимостей

Здесь, если крейт зависит от `foo` и не отказывается явно от feature-флагов по умолчанию, он также скомпилирует зависимость `syn` крейта `foo`. В свою очередь, `syn` будет собран только с тремя перечисленными feature-флагами и

никакими другими. Отказ от feature-флагов по умолчанию таким образом и подключение только того, что нужно, — отличный способ сократить время компиляции!

ОПЦИОНАЛЬНЫЕ ЗАВИСИМОСТИ КАК FEATURE-ФЛАГИ (OPTIONAL DEPENDENCIES AS FEATURES)

Когда вы определяете feature-флаг, список, следующий за знаком равенства, сам по себе является списком feature-флагов. Поначалу это может прозвучать немного странно — в листинге 6-3 `syn` является зависимостью, а не feature-флагом. Оказывается, Cargo делает каждую опциональную зависимость feature-флагом с тем же именем, что и у зависимости. Вы убедитесь в этом, если попытаетесь добавить feature-флаг с тем же именем, что и опциональная зависимость; Cargo этого не позволит. Поддержка отдельных пространств имён для feature-флагов и зависимостей в Cargo находится в разработке, но на момент написания ещё не стабилизирована. А пока, если вам нужен feature-флаг, названный по имени зависимости, вы можете переименовать зависимость, используя `package = ""`, чтобы избежать конфликта имён. Список feature-флагов, которые включает feature-флаг, может также включать feature-флаги зависимостей. Например, вы можете написать `derive = ["syn/derive"]`, чтобы ваш feature-флаг `derive` включал feature-флаг `derive` зависимости `syn`.

Использование feature-флагов в вашем крейте (Using Features in Your Crate)

При использовании feature-флагов вам нужно убедиться, что ваш код использует зависимость только в том случае, если она доступна. А если ваш feature-флаг включает определённый компонент, вам нужно убедиться, что если feature-флаг не включён, то компонент не подключается.

Этого вы добиваетесь с помощью условной компиляции (conditional compilation), которая позволяет задавать условия, при которых тот или иной фрагмент кода должен или не должен компилироваться. Условная компиляция в основном выражается с помощью атрибута `#[cfg]`. Существует также тесно связанный с ним макрос `cfg!`, который позволяет менять поведение во время выполнения на основе аналогичных условий. С условной компиляцией можно делать множество изящных вещей, как мы увидим далее

в этой главе, но самая базовая форма — это `#[cfg(feature = "some-feature")]`, благодаря которой следующая «вещь» в исходном коде компилируется только в том случае, если включён feature-флаг `some-feature`. Аналогично, `if cfg!(feature = "some-feature")` эквивалентно `if true`, только если включён feature-флаг `derive` (и `if false` в противном случае).

Атрибут `#[cfg]` используется чаще, чем макрос `cfg!`, потому что макрос модифицирует поведение во время выполнения на основе feature-флага, из-за чего бывает трудно гарантировать аддитивность feature-флагов. Вы можете размещать `#[cfg]` перед определёнными *элементами* Rust — такими как функции и определения типов, блоки `impl`, модули и инструкции `use`, — а также перед некоторыми другими конструкциями, такими как поля структур, аргументы функций и инструкции. Однако атрибут `#[cfg]` нельзя ставить где угодно; место, где он может появляться, тщательно ограничено командой разработчиков языка Rust, чтобы условная компиляция не могла создавать слишком странных и трудных для отладки ситуаций.

Помните, что модификация определённых публичных частей вашего API может непреднамеренно сделать его неисчерпывающим (*nonadditive*), что, в свою очередь, может сделать невозможной компиляцию вашего крейта для некоторых пользователей. Здесь часто можно использовать правила обратно совместимых изменений как ориентир — например, если вы делаете вариант перечисления или публичное поле структуры зависящим от feature-флага, то этот тип также должен быть помечен `#[non_exhaustive]`. В противном случае зависимый крейт, у которого этот feature-флаг не включён, может перестать компилироваться, если этот feature-флаг будет включён из-за какого-то второго крейта в дереве зависимостей.

ПРИМЕЧАНИЕ Если вы пишете большой крейт, в котором ожидаете, что вашим пользователям понадобится лишь подмножество функциональности, рассмотрите возможность сделать так, чтобы более крупные компоненты (обычно модули) были защищены feature-флагами. Тогда пользователи смогут подключать и оплачивать стоимость компиляции только тех частей, которые им действительно нужны.

Рабочие пространства (Workspaces)

Крейты играют множество ролей в Rust — они являются вершинами в графе зависимостей, границами для когерентности трейтов и областями действия для feature-флагов компиляции. Из-за этого каждый крейт управляется как единая единица компиляции; компилятор Rust обрабатывает крейт более или менее как один большой исходный файл, компилируемый как единый блок, который в конечном итоге превращается в один бинарный вывод (либо бинарный файл, либо библиотеку).

Хотя это упрощает многие аспекты работы компилятора, это также означает, что с большими крейтами может быть неприятно работать. Если вы изменяете юнит-тест, комментарий или тип в одной части приложения, компилятор должен заново проанализировать весь крейт, чтобы определить, что (если вообще что-то) изменилось. Внутри компилятор реализует ряд механизмов для ускорения этого процесса, таких как инкрементальная перекомпиляция и параллельная генерация кода, но в конечном итоге размер вашего крейта является важным фактором того, как долго будет компилироваться ваш проект.

По этой причине по мере роста проекта вы можете захотеть разбить его на несколько крейтов, которые внутренне зависят друг от друга. У Cargo есть как раз та функция, которая делает это удобным: рабочие пространства. Рабочее пространство (workspace) — это набор крейтов (часто называемых *подкрейтами*, subcrates), которые связаны вместе верхнеуровневым файлом `Cargo.toml`, подобным показанному в листинге 6-4.

```
[workspace]
members = [
  "foo",
  "bar/one",
  "bar/two",
]
```

Листинг 6-4: Файл `Cargo.toml` рабочего пространства

Массив `members` — это список каталогов, каждый из которых содержит крейт рабочего пространства. У всех этих крейтов есть свои собственные файлы `Cargo.toml` в их собственных подкаталогах, но они совместно используют один файл `Cargo.lock` и один каталог вывода. Имена крейтов не обязаны совпадать с записью в `members`. Распространено, но не обязательно, что крейты в рабочем

пространстве используют общий префикс имени, обычно выбранный по имени «главного» крейта. Например, в крейте `tokio` участники называются `tokio`, `tokio-test`, `tokio-macros` и так далее.

Пожалуй, самая важная особенность рабочих пространств состоит в том, что вы можете взаимодействовать со всеми участниками рабочего пространства, вызывая `cargo` в корне рабочего пространства. Хотите проверить, что все они компилируются? `cargo check` проверит их все. Хотите запустить все ваши тесты? `cargo test` протестирует их все. Это не настолько удобно, как иметь всё в одном крейте, поэтому не стоит разбивать всё на крошечные крейты, но это довольно хорошее приближение.

ПРИМЕЧАНИЕ Команды Cargo, как правило, делают «правильную вещь» в рабочем пространстве. Если вам когда-либо понадобится устранить неоднозначность, например если два крейта рабочего пространства оба содержат бинарный файл с одним и тем же именем, используйте флаг `-p` (для `package`). Если вы находитесь в подкаталоге конкретного крейта рабочего пространства, вы можете передать `--workspace`, чтобы выполнить команду для всего рабочего пространства.

Как только у вас есть `Cargo.toml` уровня рабочего пространства с массивом участников рабочего пространства, вы можете настроить свои крейты так, чтобы они зависели друг от друга, используя зависимости по пути (`path dependencies`), как показано в листинге 6-5.

```
# bar/two/Cargo.toml
[dependencies]
one = { path = "../one" }
# bar/one/Cargo.toml
[dependencies]
foo = { path = "../../foo" }
```

Листинг 6-5: Межкрейтовые зависимости среди крейтов рабочего пространства

Теперь если вы внесёте изменение в крейт `bar/two`, то только этот крейт будет перекомпилирован, поскольку `foo` и `bar/one` не изменились. Компиляция вашего проекта с нуля может даже стать быстрее, поскольку компилятору не нужно анализировать весь исходный код вашего проекта в поисках возможностей оптимизации.

ЗАДАНИЕ ВНУТРИРАБОЧИХ ЗАВИСИМОСТЕЙ (SPECIFYING INTRA-WORKSPACE DEPENDENCIES)

Самый очевидный способ указать, что один крейт в рабочем пространстве зависит от другого, — использовать спецификатор пути (path), как показано в листинге 6-5. Однако если ваши отдельные подкрейты предназначены для публичного использования, вы можете захотеть использовать вместо этого спецификаторы версий (version).

Допустим, у вас есть крейт, который зависит от Git-версии того самого крейта `one` из рабочего пространства `bar` в листинге 6-5 через `one = { git = ". . ." }`, а также от выпущенной версии `foo` (тоже из `bar`) через `foo = "1.0.0"`. Cargo послушно загрузит Git-репозиторий `one`, который содержит всё рабочее пространство `bar`, и увидит, что `one`, в свою очередь, зависит от `foo`, расположенного по адресу `../../foo` внутри рабочего пространства. Но Cargo не знает, что выпущенная версия `foo = "1.0.0"` и `foo` в Git-репозитории — это один и тот же крейт! Он считает их двумя отдельными зависимостями, у которых просто оказалось одинаковое имя.

Возможно, вы уже видите, к чему это ведёт. Если вы попытаетесь использовать какой-либо тип из `foo (1.0.0)` с API из `one`, принимающим тип из `foo`, компилятор отвергнет код. Несмотря на то что типы имеют одинаковое имя, компилятор не может знать, что они являются одним и тем же базовым типом. И пользователь будет совершенно сбит с толку, поскольку компилятор скажет что-то вроде «`expected foo::Type, got foo::Type`».

Лучший способ смягчить эту проблему — использовать зависимости по пути между подкрейтами только в том случае, если они зависят от неопубликованных изменений. Пока работаешь с `foo 1.0.0`, следует указывать `foo = "1.0.0"` в его зависимостях. Только если вы внесёте в `foo` изменение, которое нужно `one`, вам следует изменить `one` так, чтобы он использовал зависимость по пути. И как только вы выпустите новую версию `foo`, от которой `one` может зависеть, вам следует снова убрать зависимость по пути.

У этого подхода тоже есть свои недостатки. Теперь если вы измените `foo`, а затем запустите тесты для `one`, вы увидите, что `one` будет протестирован с использованием старой версии `foo`, что может

оказаться не тем, чего вы ожидали. Вероятно, вы захотите настроить инфраструктуру непрерывной интеграции так, чтобы тестировать каждый подкрейт как с последними выпущенными версиями других подкрейтов, так и со всеми ними, настроенными на использование зависимостей по пути.

Конфигурация проекта (Project Configuration)

Запуск `cargo new` создаёт для вас минимальный `Cargo.toml`, который содержит имя крейта, его номер версии, некоторую информацию об авторе и пустой список зависимостей. Это позволит вам зайти достаточно далеко, но по мере того как ваш проект взрослеет, есть ряд полезных вещей, которые вы можете захотеть добавить в свой `Cargo.toml`.

Метаданные крейта (Crate Metadata)

Первое и самое очевидное, что нужно добавить в ваш `Cargo.toml`, — это все директивы метаданных, которые поддерживает Cargo. Помимо очевидных полей вроде `description` и `homepage`, бывает полезно включить такую информацию, как путь к файлу README крейта (`readme`), бинарный файл, запускаемый по умолчанию при `cargo run` (`default-run`), и дополнительные `keywords` и `categories`, помогающие `crates.io` категоризировать ваш крейт.

Для крейтов с более запутанной структурой проекта также полезно задать поля метаданных `include` и `exclude`. Они определяют, какие файлы должны быть включены и опубликованы в составе вашего пакета. По умолчанию Cargo включает все файлы в каталоге крейта, кроме перечисленных в файле `.gitignore`, но это может быть не тем, что вам нужно, если у вас есть, например, большие тестовые фикстуры, не относящиеся к проекту скрипты или другие вспомогательные данные в том же каталоге, которые вы *хотите* держать под контролем версий. Как следует из их названий, `include` и `exclude` позволяют включать только определённый набор файлов или исключать файлы, соответствующие заданному набору шаблонов, соответственно.

ПРИМЕЧАНИЕ Если у вас есть крейт, который никогда не должен публиковаться или должен публиковаться только в определённых альтернативных реестрах (то есть *не* на `crates.io`), вы можете установить директиву `publish` в `false` или указать список разрешённых

реестров.

Список директив метаданных, которые вы можете использовать, продолжает расти, поэтому периодически заглядывайте на страницу Manifest Format в справочнике Cargo (<https://doc.rust-lang.org/cargo/reference/manifest.html>).

Конфигурация сборки (Build Configuration)

`Cargo.toml` также может дать вам контроль над тем, как Cargo собирает ваш крейт. Самый очевидный инструмент для этого — параметр `build`, который позволяет написать полностью пользовательскую программу сборки для вашего крейта (мы вернёмся к этому в главе 12). Однако Cargo также предоставляет два меньших, но очень полезных механизма, которые мы здесь рассмотрим: патчи (`patches`) и профили (`profiles`).

[patch]

Секция `[patch]` файла `Cargo.toml` позволяет указать другой источник для зависимости, который вы можете использовать временно, независимо от того, где в дереве зависимостей появляется пропатченная зависимость. Это бесценно, когда вам нужно скомпилировать ваш крейт с модифицированной версией какой-нибудь транзитивной зависимости, чтобы протестировать исправление ошибки, улучшение производительности или новый минорный релиз, который вы собираетесь опубликовать. Листинг 6-6 показывает пример того, как можно временно использовать вариант набора зависимостей.

```
[patch.crates-io]
# использовать локальный (предположительно модифицированный) источник
regex = { path = "/home/jon/regex" }
# использовать модификацию из git-ветки
serde = { git = "https://github.com/serde-rs/serde.git", branch = "faster" }
# пропатчить git-зависимость
[patch.'https://github.com/jonhoo/project.git']
project = { path = "/home/jon/project" }
```

Листинг 6-6: Переопределение источников зависимостей в Cargo.toml с помощью [patch]

Даже если вы патчите зависимость, Cargo позаботится о проверке версий крейта, чтобы вы случайно не пропатчили неправильную мажорную версию крейта. Если вы по какой-то причине транзитивно зависите от нескольких

мажорных версий одного и того же крейта, вы можете пропатчить каждую из них, дав им различные идентификаторы, как показано в листинге 6-7.

```
[patch.crates-io]
nom4 = { path = "/home/jon/nom4", package = "nom" }
nom5 = { path = "/home/jon/nom5", package = "nom" }
```

Листинг 6-7: Переопределение нескольких версий одного и того же крейта в Cargo.toml с помощью [patch]

Cargo заглянет в Cargo.toml внутри каждого пути, поймёт, что /nom4 содержит мажорную версию 4, а /nom5 — мажорную версию 5, и пропатчит обе версии соответствующим образом. Ключевое слово package сообщает Cargo, что в обоих случаях нужно искать крейт по имени nom вместо использования идентификатора зависимости (того, что слева), как он делает это по умолчанию. Вы можете использовать package таким образом и в ваших обычных зависимостях, чтобы переименовать зависимость!

Имейте в виду, что патчи не учитываются в пакете, который загружается при публикации крейта. Крейт, который зависит от вашего крейта, будет использовать только свою собственную секцию [patch] (которая может быть пустой), а не секцию вашего крейта!

КРЕЙТЫ ПРОТИВ ПАКЕТОВ (CRATES VS. PACKAGES) Возможно, вам интересно, в чём разница между пакетом (package) и крейтом (crate). Эти два термина часто используются как взаимозаменяемые в неформальном контексте, но у них также есть конкретные определения, которые различаются в зависимости от того, говорите ли вы о компиляторе Rust, Cargo, crates.io или о чём-то ещё. Лично я представляю крейт как иерархию модулей Rust, начинающуюся с корневого файла .rs (того, в котором можно использовать атрибуты уровня крейта вроде #![feature]) — обычно это что-то вроде lib.rs или main.rs. В отличие от этого, пакет — это набор крейтов и метаданных, то есть, по сути, всё то, что описывается файлом Cargo.toml. Сюда могут входить библиотечный крейт, несколько бинарных крейтов, несколько крейтов интеграционных тестов и, возможно, даже несколько участников рабочего пространства, у которых самих есть файлы Cargo.toml.

[profile]

Секция `[profile]` позволяет передавать дополнительные опции компилятору Rust, чтобы изменить способ компиляции вашего крейта. Эти опции в основном делятся на три категории: опции производительности, опции отладки и опции, изменяющие поведение кода определёнными пользователем способами. У всех у них разные значения по умолчанию в зависимости от того, компилируете ли вы в режиме отладки (debug) или релиза (release) (существуют и другие режимы).

Три основные опции производительности — это `opt-level`, `codegen-units` и `lto`. Опция `opt-level` настраивает производительность во время выполнения, указывая компилятору, насколько агрессивно оптимизировать вашу программу (0 — «совсем нет», 3 — «насколько возможно»). Чем выше значение, тем более оптимизированным будет ваш код, что *может* сделать его работу быстрее. Однако дополнительная оптимизация достигается ценой более длительного времени компиляции, поэтому оптимизации обычно включаются только для релизных сборок.

ПРИМЕЧАНИЕ Вы также можете установить `opt-level` в "s", чтобы оптимизировать по размеру бинарного файла, что может быть важно на встраиваемых платформах.

Опция `codegen-units` касается производительности во время компиляции. Она указывает компилятору, на сколько независимых задач компиляции (*единиц генерации кода*, code generation units) ему разрешено разбить компиляцию одного крейта. Чем на большее число частей разбивается компиляция большого крейта, тем быстрее он скомпилируется, поскольку больше потоков смогут помочь скомпилировать крейт параллельно. К сожалению, для достижения этого ускорения потоки должны работать более или менее независимо, что означает, что оптимизация кода страдает. Представьте, например, что сегмент крейта, компилируемый в одном потоке, мог бы выиграть от встраивания (inlining) какого-то кода из другого сегмента — поскольку эти два сегмента независимы, такое встраивание невозможно! Таким образом, эта настройка — компромисс между производительностью во время компиляции и производительностью во время выполнения. По умолчанию Rust использует фактически неограниченное число единиц генерации кода в режиме отладки (по сути, «компилируй как можно быстрее») и меньшее число (16 на момент написания) в режиме релиза.

Настройка `lto` переключает оптимизацию во время компоновки (*link-time optimization*, LTO), которая заставляет компилятор (или компоновщик, если уж совсем технически) совместно оптимизировать фрагменты вашей программы, известные как *единицы компиляции* (compilation units), которые изначально были скомпилированы по отдельности. Точные детали LTO выходят за рамки этой книги, но основная идея заключается в том, что вывод каждой единицы компиляции содержит информацию о коде, который в неё вошёл. После того как все единицы скомпилированы, компоновщик делает ещё один проход по всем единицам и использует дополнительную информацию для оптимизации объединённого скомпилированного кода. Этот дополнительный проход увеличивает время компиляции, но возвращает большую часть производительности во время выполнения, которая могла быть потеряна из-за разбиения компиляции на меньшие части. В частности, LTO может дать значительный прирост производительности для чувствительных к производительности программ, которые могут выиграть от межкрейтовой оптимизации. Однако имейте в виду, что межкрейтовый LTO может существенно увеличить время компиляции.

Rust по умолчанию выполняет LTO по всем единицам генерации кода внутри каждого крейта в попытке компенсировать потерянные оптимизации, вызванные использованием множества единиц генерации кода. Поскольку LTO выполняется только внутри каждого крейта, а не между крейтами, этот дополнительный проход не слишком обременителен, и добавленное время компиляции должно быть меньше, чем время, сэкономленное за счёт использования большого числа единиц генерации кода. Rust также предлагает технику, известную как *тонкий LTO* (thin LTO), которая позволяет распараллелить проход LTO в большей степени, ценой пропуска некоторых оптимизаций, которые нашёл бы «полный» проход LTO.

ПРИМЕЧАНИЕ LTO во многих случаях также можно использовать для оптимизации через границы интерфейса внешних функций (foreign function interface). Подробнее см. флаг `rustc linker-plugin-lto`.

Секция `[profile]` также поддерживает флаги, которые помогают в отладке, такие как `debug`, `debug-assertions` и `overflow-checks`. Флаг `debug` указывает компилятору включить отладочные символы в скомпилированный бинарный файл. Это увеличивает размер бинарного файла, но означает, что вы получаете имена функций и тому подобное, а не просто адреса инструкций в

трассировках стека и профилях. Флаг `debug-assertions` включает макрос `debug_assert!` и другой связанный отладочный код, который иначе не компилируется (через `cfg(debug_assertions)`). Такой код может замедлять вашу программу, но облегчает выявление сомнительного поведения во время выполнения. Флаг `overflow-checks`, как следует из названия, включает проверки переполнения для целочисленных операций. Это замедляет работу (замечаете ли вы здесь тенденцию?), но может помочь выявить хитрые ошибки на ранней стадии. По умолчанию всё это включено в режиме отладки и отключено в режиме релиза.

[profile.*.panic]

У секции `[profile]` есть ещё один флаг, заслуживающий собственного подраздела: `panic`. Эта опция определяет, что происходит, когда код в вашей программе вызывает `panic!`, прямо или косвенно через что-то вроде `unwrap`. Вы можете установить `panic` либо в `unwind` (значение по умолчанию на большинстве платформ), либо в `abort`. Мы поговорим о панике и раскрутке стека (`unwinding`) в главе 10, но здесь я дам краткое резюме.

Обычно в Rust, когда ваша программа паникует, поток начинает *раскручивать* (`unwinding`) свой стек. Раскрутку стека можно представить как принудительный рекурсивный возврат из текущей функции вплоть до самого низа стека этого потока. То есть если `main` вызвал `foo`, `foo` вызвал `bar`, а `bar` вызвал `baz`, то паника в `baz` принудительно вернёт управление из `baz`, затем из `bar`, затем из `foo` и, наконец, из `main`, что приведёт к завершению программы. Поток, который раскручивается, нормально освободит все значения на стеке, что даёт этим значениям возможность очистить ресурсы, сообщить об ошибках и так далее. Это даёт работающей системе возможность корректно завершиться даже в случае паники.

Когда поток паникует и раскручивается, другие потоки продолжают работать незатронутыми. Только когда (и если) завершается поток, в котором выполнялась `main`, завершается программа. То есть паника обычно изолирована в потоке, в котором она произошла.

Это означает, что раскрутка стека — обоюдоострый меч: программа продолжает прихрамывать с некоторыми отказавшими компонентами, что может привести ко всевозможным странным поведениям. Например, представьте поток, который паникует на полпути обновления состояния в

Mutex. Любой поток, который впоследствии захватит этот Mutex, теперь должен быть готов справиться с тем фактом, что состояние может находиться в частично обновлённом, несогласованном виде. По этой причине некоторые примитивы синхронизации (вроде Mutex) запоминают, произошла ли паника во время их последнего доступа, и сообщают об этом любому потоку, который пытается обратиться к примитиву впоследствии. Если поток сталкивается с таким состоянием, он обычно тоже паникует, что приводит к каскаду, который в конечном итоге завершает всю программу. Но это, пожалуй, лучше, чем продолжать работу с повреждённым состоянием!

Учёт, необходимый для поддержки раскрутки стека, не бесплатен и часто требует специальной поддержки со стороны компилятора и целевой платформы. Например, многие встраиваемые платформы вообще не могут эффективно раскручивать стек. Поэтому Rust поддерживает другой режим паники: `abort` гарантирует, что вся программа просто завершается немедленно при возникновении паники. В этом режиме ни один поток не выполняет никакой очистки. Это может показаться суровым, и так оно и есть, но это гарантирует, что программа никогда не работает в полурботающем состоянии и что ошибки становятся видны немедленно.

ПРЕДУПРЕЖДЕНИЕ Настройка `panic` глобальна — если вы установите её в `abort`, все ваши зависимости также будут скомпилированы с `abort`.

Возможно, вы заметили, что когда поток паникует, он обычно печатает *обратную трассировку* (backtrace): след вызовов функций, который привёл к месту, где произошла паника. Это тоже форма раскрутки стека, хотя она отделена от поведения раскрутки при панике, обсуждавшегося здесь. Вы можете получать обратные трассировки даже с `panic=abort`, передав `-Cforce-unwind-tables` в `rustc`, что заставляет `rustc` включать информацию, необходимую для прохода вверх по стеку, при этом всё равно завершая программу при панике.

ПЕРЕОПРЕДЕЛЕНИЯ ПРОФИЛЯ (PROFILE OVERRIDES) Вы можете задать опции профиля только для конкретной зависимости или для конкретного профиля, используя *переопределения профиля* (profile overrides). Например, листинг 6-8 показывает, как включить агрессивные оптимизации для крейта `serde` и умеренные оптимизации для всех остальных крейтов в режиме отладки, используя синтаксис


```
[profile.<profile-name>.package.<crate-name>].
```

```
``toml [profile.dev.package.serde] opt-level = 3 [profile.dev.package."*"]  
opt-level = 2 ``
```

Листинг 6-8: Переопределение опций профиля для конкретной зависимости или для конкретного режима

Такого рода переопределение оптимизации может оказаться удобным, если какая-то зависимость была бы недопустимо медленной в режиме отладки (например, распаковка или видеокодирование), и вам нужно её оптимизировать, чтобы ваш набор тестов не выполнялся несколько дней. Вы можете также задать глобальные значения профиля по умолчанию, используя секцию [profile.dev] (или подобную) в файле конфигурации Cargo в ~/.cargo/config.

Когда вы задаёте параметры оптимизации для конкретной зависимости, имейте в виду, что параметры применяются только к коду, скомпилированному как часть этого крейта; если у serde в этом примере есть обобщённый метод или тип, который вы используете в своём крейте, код этого метода или типа будет мономорфизирован и оптимизирован в *вашем* крейте, и применятся настройки профиля *вашего* крейта, а не те, что в переопределении профиля для serde.

Условная компиляция (Conditional Compilation)

Большая часть кода на Rust, который вы пишете, универсальна — он будет работать одинаково независимо от того, на каком CPU или операционной системе он выполняется. Но иногда вам придётся делать что-то особенное, чтобы код заработал в Windows, на чипах ARM или при компиляции под определённый платформенный двоичный интерфейс приложения (ABI). Или, может быть, вам нужна оптимизированная версия конкретной функции, когда доступна определённая инструкция CPU, или нужно отключить медленный, но неинтересный код настройки при работе в среде непрерывной интеграции (CI). Для таких случаев Rust предоставляет механизмы *условной компиляции* (conditional compilation), при которой определённый сегмент кода компилируется только в том случае, если определённые условия истинны для среды компиляции.

Мы обозначаем условную компиляцию ключевым словом `cfg`, которое вы видели в разделе «Использование feature-флагов в вашем крейте». Обычно оно появляется в форме атрибута `#[cfg(condition)]`, который говорит компилировать следующий элемент только в том случае, если условие истинно. У Rust также есть `#[cfg_attr(condition, attribute)]`, который компилируется как `#[attribute]`, если условие выполняется, и не имеет эффекта в противном случае. Вы также можете вычислить условие `cfg` как булево выражение, используя макрос `cfg!(condition)`.

Каждая конструкция `cfg` принимает единственное условие, состоящее из опций вроде `feature = "some-feature"`, и комбинаторов `all`, `any` и `not`, которые делают то, чего вы, вероятно, ожидаете. Опции — это либо простые имена, вроде `unix`, либо пары ключ/значение, как у feature-условий.

Существует ряд интересных опций, от которых вы можете сделать зависимой компиляцию. Давайте пройдемся по ним, от наиболее распространённых к наименее распространённым:

Опции feature-флагов

Примеры этих опций вы уже видели. Опции feature-флагов имеют форму `feature = "name-of-feature"` и считаются истинными, если названный feature-флаг включён. Вы можете проверять несколько feature-флагов в одном условии с помощью комбинаторов. Например, `any(feature = "f1", feature = "f2")` истинно, если включён либо feature-флаг `f1`, либо feature-флаг `f2`.

Опции операционной системы

Они используют синтаксис ключ/значение с ключом `target_os` и значениями вроде `windows`, `macos` и `linux`. Вы также можете указать семейство операционных систем с помощью `target_family`, который принимает значение `windows` или `unix`. Эти значения достаточно распространены, поэтому получили собственные именованные сокращённые формы, так что вы можете использовать `cfg(windows)` и `cfg(unix)` напрямую. Например, если бы вы хотели, чтобы определённый сегмент кода компилировался только в macOS и Windows, вы бы написали `#[cfg(any(windows, target_os = "macos"))]`.

Опции контекста

Они позволяют адаптировать код под конкретный контекст компиляции.

Самая распространённая из них — опция `test`, которая истинна только тогда, когда крейт компилируется под тестовым профилем. Имейте в виду, что `test` устанавливается только для тестируемого крейта, но не для каких-либо его зависимостей. Это также означает, что `test` не установлен в вашем крейте при запуске интеграционных тестов; интеграционные тесты компилируются под тестовым профилем, тогда как ваш фактический крейт компилируется обычным образом (то есть без установленного `test`). То же самое относится к опциям `doc` и `doctest`, которые устанавливаются только при сборке документации или компиляции doc-тестов соответственно. Есть также опция `debug_assertions`, которая по умолчанию установлена в режиме отладки.

Опции инструментов

Некоторые инструменты, такие как `clippy` и `Miri`, устанавливают пользовательские опции (подробнее об этом позже), которые позволяют кастомизировать компиляцию при запуске под этими инструментами. Обычно эти опции называются по имени соответствующего инструмента. Например, если вы хотите, чтобы конкретный вычислительно затратный тест не запускался под `Miri`, вы можете дать ему атрибут `#[cfg_attr(miri, ignore)]`.

Опции архитектуры

Они позволяют компилировать на основе набора инструкций CPU, на который ориентируется компилятор. Вы можете указать конкретную архитектуру с помощью `target_arch`, который принимает значения вроде `x86`, `mips` и `aarch64`, или указать конкретную возможность платформы с помощью `target_feature`, который принимает значения вроде `avx` или `sse2`. Для очень низкоуровневого кода вам также могут пригодиться опции `target_endian` и `target_pointer_width`.

Опции компилятора

Они позволяют адаптировать ваш код под платформенный ABI, под который он компилируется, и доступны через `target_env` со значениями вроде `gnu`, `msvc` и `musl`. По историческим причинам это значение часто пусто, особенно на GNU-платформах. Обычно эта опция нужна вам только в том случае, если вам нужно напрямую взаимодействовать с ABI среды, например при компоновке с ABI-специфичным именем символа с помощью `#[link]`.

Хотя условия `cfg` обычно используются для кастомизации кода, некоторые из

них можно также использовать для кастомизации зависимостей. Например, зависимость `winrt` обычно имеет смысл только в Windows, а крейт `nix`, вероятно, полезен только на платформах на основе Unix. Листинг 6-9 даёт пример того, как можно использовать условия `cfg` для этого:

```
[target.'cfg(windows)'.dependencies]
winrt = "0.7"
[target.'cfg(unix)'.dependencies]
nix = "0.17"
```

Листинг 6-9: Условные зависимости

Здесь мы указываем, что `winrt` версии 0.7 следует считать зависимостью только при `cfg(windows)` (то есть в Windows), а `nix` версии 0.17 — только при `cfg(unix)` (то есть в Linux, macOS и других платформах на основе Unix). Следует иметь в виду, что секция `[dependencies]` вычисляется очень рано в процессе сборки, когда доступны только определённые опции `cfg`. В частности, `feature-опции` и опции контекста в этот момент ещё недоступны, поэтому вы не можете использовать этот синтаксис для подтягивания зависимостей на основе `feature-флагов` и контекстов. Однако вы можете использовать любой `cfg`, который зависит только от спецификации целевой платформы или архитектуры, а также любые опции, явно установленные инструментами, вызывающими `rustc` (вроде `cfg(miri)`).

ПРИМЕЧАНИЕ Раз уж мы заговорили о спецификациях зависимостей, я настоятельно рекомендую настроить инфраструктуру CI так, чтобы выполнять базовый аудит ваших зависимостей с помощью инструментов вроде `cargo-deny` и `cargo-audit`. Эти инструменты обнаружат случаи, когда вы транзитивно зависите от нескольких мажорных версий одной зависимости, когда вы зависите от крейтов, которые не поддерживаются или имеют известные уязвимости безопасности, или когда вы используете лицензии, которых, возможно, хотели бы избежать. Использование такого инструмента — отличный способ автоматизированно повысить качество вашей кодовой базы!

Добавить собственные пользовательские опции условной компиляции тоже довольно просто. Вам всего лишь нужно убедиться, что `--cfg=myoption` передаётся в `rustc`, когда `rustc` компилирует ваш крейт. Проще всего это сделать, добавив ваш `--cfg` в переменную окружения `RUSTFLAGS`. Это может

пригодиться в CI, где вы можете захотеть кастомизировать набор тестов в зависимости от того, выполняется ли он на CI или на машине разработчика: добавьте `--cfg=ci` в RUSTFLAGS в настройке CI, а затем используйте `cfg(ci)` и `cfg(not(ci))` в вашем коде. Опции, заданные таким образом, также доступны в зависимостях Cargo.toml.

Версионирование (Versioning)

Все крейты Rust имеют версии и, как ожидается, следуют реализации семантического версионирования (semantic versioning) в Cargo. *Семантическое версионирование* диктует правила того, какие виды изменений требуют каких видов увеличения версии и какие версии считаются совместимыми, и каким образом. Сам стандарт RFC 1105 стоит почитать (он не слишком технический), но, если вкратце, он различает три вида изменений: ломающие изменения, которые требуют увеличения мажорной версии; добавления, которые требуют изменения минорной версии; и исправления ошибок, которые требуют только изменения patch-версии. RFC 1105 неплохо справляется с описанием того, что составляет ломающее изменение в Rust, и мы затрагивали некоторые из этих аспектов в других местах этой книги.

Я не буду здесь углубляться в точную семантику различных видов изменений. Вместо этого я хочу подчеркнуть некоторые менее очевидные способы, которыми номера версий возникают в экосистеме Rust и которые вам нужно иметь в виду при принятии решения о том, как версионировать свои собственные крейты.

Минимальная поддерживаемая версия Rust (Minimum Supported Rust Version)

Первая «русто-особенность» — это *минимальная поддерживаемая версия Rust* (minimum supported Rust version, MSRV). В сообществе Rust ведётся много споров о том, какой политики следует придерживаться проектам, когда речь идёт о MSRV и версионировании, и по-настоящему хорошего ответа нет. Суть проблемы в том, что некоторые пользователи Rust ограничены использованием более старых версий Rust, часто в корпоративной среде, где у них мало выбора. Если мы постоянно используем преимущества недавно стабилизированных API, эти пользователи не смогут компилировать последние

версии наших крейтов и останутся позади.

Есть два приёма, которые авторы крейтов могут использовать, чтобы немного облегчить жизнь пользователям в таком положении. Первый — установить политику MSRV, обещающую, что новые версии крейта всегда будут компилироваться с любым стабильным релизом за последние X месяцев. Точное число варьируется, но часто это 6 или 12 месяцев. С шестинедельным циклом релизов Rust это соответствует четырём или восьми последним стабильным релизам соответственно. Любой новый код, добавляемый в проект, должен компилироваться компилятором MSRV (обычно это проверяется CI) или быть отложенным, пока политика MSRV не позволит его влить. Иногда это может быть болезненно, поскольку означает, что эти крейты не могут использовать самые новейшие возможности языка, но это облегчит жизнь вашим пользователям.

Второй приём — обязательно увеличивать номер минорной версии вашего крейта каждый раз, когда меняется MSRV. Так, если вы выпускаете версию 2.7.0 и она повышает ваш MSRV с Rust 1.44 до Rust 1.45, то проект, застрявший на 1.44 и зависящий от вашего крейта, может использовать спецификатор версии зависимости `version = "2, <2.7"`, чтобы проект продолжал работать, пока не сможет перейти на Rust 1.45. Важно увеличивать именно минорную версию, а не только patch-версию, чтобы вы всё ещё могли выпускать критические исправления безопасности для предыдущего MSRV-релиза, делая ещё один patch-релиз при необходимости.

Некоторые проекты относятся к поддержке MSRV настолько серьёзно, что считают изменение MSRV ломающим изменением и увеличивают мажорный номер версии. Это означает, что нижестоящие проекты должны будут явно согласиться на изменение MSRV, а не отказываться от него — но это также означает, что пользователи, у которых нет таких строгих требований к MSRV, не увидят будущих исправлений ошибок без обновления, что, в свою очередь, может потребовать от *них* тоже выпустить ломающее изменение. Как я уже сказал, ни одно из этих решений не лишено недостатков.

Обеспечение MSRV в экосистеме Rust сегодня является непростой задачей. Лишь небольшое подмножество крейтов предоставляет какие-либо гарантии MSRV, и даже если ваши зависимости их предоставляют, вам придётся постоянно следить за ними, чтобы знать, когда они повышают свой MSRV. Когда это произойдёт, вам нужно будет сделать новый релиз вашего крейта с

ограниченными границами версий, упомянутыми ранее, чтобы убедиться, что ваш MSRV тоже не изменится. Это, в свою очередь, может вынудить вас отказаться от обновлений безопасности и производительности ваших зависимостей, поскольку вам придётся продолжать использовать более старые версии до тех пор, пока ваша политика MSRV не позволит обновление. И это решение также переносится на ваших зависимых. Были предложения встроить проверку MSRV в сам Cargo, но на момент написания ничего работоспособного не было стабилизировано.

Минимальные версии зависимостей (Minimal Dependency Versions)

Когда вы впервые добавляете зависимость, не всегда понятно, какой спецификатор версии следует ей задать. Программисты обычно выбирают последнюю версию или просто текущую мажорную версию, но велики шансы, что оба этих выбора неправильны. Под «неправильны» я не имею в виду, что ваш крейт не скомпилируется, а скорее то, что такой выбор может создать проблемы пользователям вашего крейта в будущем. Давайте рассмотрим, чем плох каждый из этих случаев.

Сначала рассмотрим случай, когда вы добавляете зависимость на `hugs = "1.7.3"`, последнюю опубликованную версию. Теперь представьте, что некий разработчик где-то зависит от вашего крейта, но он также зависит от какого-то другого крейта, `foo`, который сам зависит от `hugs`. Далее представьте, что автор `foo` очень внимателен к своей политике MSRV, поэтому он зависит от `hugs = "1, <1.6"`. Здесь вы столкнётесь с проблемой. Когда Cargo видит `hugs = "1.7.3"`, он рассматривает только версии `>=1.7`. Но затем он видит, что зависимость `foo` от `hugs` требует `<1.6`, поэтому он сдаётся и сообщает, что нет версии `hugs`, совместимой со всеми требованиями.

ПРИМЕЧАНИЕ На практике существует ряд причин, по которым крейт может явно не хотеть более новую версию зависимости. Самые распространённые — это обеспечение MSRV, удовлетворение требований корпоративного аудита (более новая версия будет содержать код, который не был проаудирован) и обеспечение воспроизводимых сборок, где используется только точно указанная версия.

Это досадно, поскольку всё могло бы прекрасно скомпилироваться, скажем, с `hugs 1.5.6`. Возможно, оно даже прекрасно скомпилируется с *любой* версией

1.X! Но, используя `hugs = "1.7.3"`, вы говорите Cargo рассматривать только версии на этой минорной версии или выше. Может, тогда решение в том, чтобы использовать вместо этого `hugs = "1"`? Нет, это тоже не совсем правильно. Может оказаться, что ваш код действительно зависит от чего-то, что было добавлено только в `hugs 1.6`, так что хотя `1.6.2` подошёл бы, `1.5.6` — нет. Вы бы этого не заметили, если бы всегда компилировали свой крейт в ситуациях, когда в итоге используется более новая версия, но если какой-то крейт в графе зависимостей укажет `hugs = "1, <1.5"`, ваш крейт не скомпилируется!

Правильная стратегия — указать самую раннюю версию, в которой есть всё, от чего зависит ваш крейт, и убедиться, что это остаётся так и по мере того, как вы добавляете новый код в свой крейт. Но как же установить эту версию, кроме как прочёсывая журналы изменений или методом проб и ошибок? Лучший вариант — использовать нестабильный флаг Cargo `-Z minimal-versions`, который заставляет ваш крейт использовать минимальную допустимую версию для всех зависимостей, а не максимальную. Затем установите все ваши зависимости просто на последний мажорный номер версии, попробуйте скомпилировать и добавьте минорную версию любым зависимостям, которые не скомпилируются. Повторяйте это до тех пор, пока всё не скомпилируется, и теперь у вас есть ваши минимальные требования к версиям!

Стоит отметить, что, как и в случае с MSRV, проверка минимальных версий сталкивается с проблемой принятия в экосистеме. Хотя вы можете правильно установить все ваши спецификаторы версий, проекты, от которых вы зависите, могут этого не сделать. Это затрудняет использование флага минимальных версий Cargo на практике (и поэтому он всё ещё нестабилен). Если вы зависите от `foo`, а `foo` зависит от `bar` со спецификатором `bar = "1"`, тогда как на самом деле ему требуется `bar = "1.4"`, Cargo сообщит, что ему не удалось скомпилировать `foo`, независимо от того, как вы указываете `foo`, потому что флаг `-Z` говорит ему всегда предпочитать минимальные версии. Вы можете обойти это, указав `bar` напрямую в *своих* зависимостях с подходящим требованием версии, но эти обходные пути могут быть болезненными в настройке и сопровождении. В итоге вы можете перечислять большое число зависимостей, которые на самом деле подтягиваются только через ваши транзитивные зависимости, и вам придётся поддерживать этот список в актуальном состоянии с течением времени.

ПРИМЕЧАНИЕ Одно из текущих предложений — представить флаг, который предпочитает минимальные версии для текущего крейта, но максимальные для зависимостей, что выглядит весьма многообещающе.

Журналы изменений (Changelogs)

Для всех крейтов, кроме самых тривиальных, я настоятельно рекомендую вести журнал изменений (changelog). Мало что более раздражает, чем увидеть, что зависимость получила мажорное обновление версии, а затем рыться в Git-логах, пытаясь выяснить, что изменилось и как обновить ваш код. Я рекомендую *не* просто сбрасывать ваши Git-логи в файл с именем *changelog*, а вместо этого вести ручной журнал изменений. Он с гораздо большей вероятностью окажется полезным.

Простой, но хороший формат для журналов изменений — формат Keep a Changelog, задокументированный по адресу <https://keepachangelog.com/>.

Невыпущенные версии (Unreleased Versions)

Rust учитывает номера версий даже тогда, когда источником зависимости является каталог или Git-репозиторий. Это означает, что семантическое версионирование важно даже тогда, когда вы ещё не выпустили релиз на crates.io; имеет значение, какая версия указана в вашем Cargo.toml между релизами. Стандарт семантического версионирования не диктует, как обрабатывать этот случай, но я предложу рабочий процесс, который работает достаточно хорошо, не будучи слишком обременительным.

После того как вы опубликовали релиз, немедленно обновите номер версии в вашем Cargo.toml до следующей patch-версии с суффиксом вроде -alpha.1. Если вы только что выпустили 2.0.3, сделайте новую версию 2.0.4-alpha.1. Если вы только что выпустили alpha, вместо этого увеличьте номер alpha.

По мере того как вы вносите изменения в код между релизами, следите за аддитивными или ломающими изменениями. Если такое происходит, а соответствующий номер версии не менялся с момента последнего релиза, увеличьте его. Например, если последняя выпущенная версия — 2.0.3, текущая версия — 2.0.4-alpha.2, и вы вносите аддитивное изменение, сделайте версию с этим изменением 2.1.0-alpha.1. Если вы внесли ломающее

изменение, она вместо этого становится 3.0.0-alpha.1. Если соответствующее увеличение версии уже было сделано, просто увеличьте номер alpha.

Когда вы делаете релиз, удалите суффикс (если только вы не хотите сделать предварительный релиз), затем публикуйте и начинайте сначала.

Этот процесс эффективен, потому что он заставляет два распространённых рабочих процесса работать намного лучше. Во-первых, представьте, что разработчик зависит от мажорной версии 2 вашего крейта, но ему нужна возможность, которая в настоящее время доступна только в Git. Затем вы вносите ломающее изменение. Если вы не увеличите мажорную версию в тот же момент, его код внезапно начнёт давать сбои неожиданным образом — либо не компилируясь, либо из-за странных проблем во время выполнения. Если же вы будете следовать описанной здесь процедуре, вместо этого Cargo уведомит его, что произошло ломающее изменение, и ему придётся либо разрешить это, либо закрепить конкретный коммит.

Затем представьте, что разработчику нужна возможность, которую он только что внёс в ваш крейт, но которая ещё не является частью какой-либо выпущенной версии вашего крейта. Он какое-то время использовал ваш крейт через Git-зависимость, поэтому другие разработчики его проекта уже имеют более старые загрузки репозитория вашего крейта. Если вы не увеличите мажорный номер версии в Git, у этого разработчика нет способа сообщить, что его проект теперь полагается на возможность, которая только что была влита. Если он отправит своё изменение, его коллеги-разработчики обнаружат, что проект больше не компилируется, поскольку Cargo повторно использует старую загрузку. Если же, с другой стороны, разработчик может увеличить минорный номер версии для Git-зависимости, то Cargo поймёт, что старая загрузка устарела.

Этот рабочий процесс ни в коей мере не идеален. Он не предоставляет хорошего способа сообщить о нескольких минорных или мажорных изменениях между релизами, и вам всё ещё нужно проделать некоторую работу, чтобы отслеживать версии. Однако он решает две из самых распространённых проблем, с которыми сталкиваются разработчики Rust, когда работают с Git-зависимостями, и даже если вы вносите несколько таких изменений между релизами, этот рабочий процесс всё равно отловит многие из проблем.

Если вы не слишком беспокоитесь о маленьких или последовательных номерах версий в релизах, вы можете улучшить этот предлагаемый рабочий процесс, просто всегда увеличивая соответствующую часть номера версии. Однако имейте в виду, что в зависимости от того, как часто вы вносите такие изменения, это может сделать ваши номера версий весьма большими!

Итоги (Summary)

В этой главе мы рассмотрели ряд механизмов для конфигурирования, организации и публикации крейтов как для вашей собственной пользы, так и для пользы других. Мы также прошли по некоторым распространённым подводным камням при работе с зависимостями и feature-флагами в Cargo, которые теперь, надеюсь, не застанут вас врасплох в будущем. В следующей главе мы обратимся к тестированию и разберёмся, как выйти за рамки простых функций `#[test]`, которые мы знаем и любим.